

Shown below is an example where a logger is created with the 'simpleExample' logger name, a console handler with the debug level set, and file handler with info level set, making sure that the debug log will not be printed. Note that both the configurations are taken from the *logging.conf* file and it's important to have the handler and logger set for 'root'.

- In file handler:
 - The *propagate* entry is set to 1 to indicate that messages must propagate to handlers higher up the logger hierarchy from this logger, or 0 to indicate that messages are not propagated to handlers up the hierarchy. The *qualname* entry is the hierarchical channel name of the logger – it is the name used by the application to get the logger.
 - In file rotate handler:
 - Here we have used file rotate handler to avoid logging all the content in one huge file. We have configured *maxBytes*=10, which means about 10 bytes written will be rotated. *backupcount*=5 indicates how many rotated files need to be kept, and then the remaining are cleaned.
3. By creating formatter with time, module name, level and message:

```
fileconfig.py
import logging
import logging.config
logging.config.fileConfig('logging.conf')
# create logger
logger1 = logging.getLogger('simpleExample')
logger = logging.getLogger()
# 'application' code
logger.debug('debug message')
logger1.info('info message')
logger1.warning('warn message')
logger1.error('error message')
logger1.critical('critical message')
```

```
loggin.conf
[loggers]
keys=root,simpleExample
[handlers]
keys=consoleHandler,simpleFileHandler
[formatters]
keys=simpleFormatter
[logger_root]
level=DEBUG
handlers=consoleHandler
[logger_simpleExample]
level=INFO
handlers=simpleFileHandler
qualname=simpleExample
```

```
propagate=0
[handler_consoleHandler]
class=StreamHandler
formatter=simpleFormatter
args=(sys.stdout,)
[handler_simpleFileHandler]
class=handlers.RotatingFileHandler
formatter=simpleFormatter
maxBytes=10
args=('test.log','a',10,5)
[formatter_simpleFormatter]
format=%(asctime)s - %(name)s - %(levelname)s - %(message)s
```

The output is:

```
$ python fileconfig.py

2021-11-28 17:47:26,858 - root - DEBUG - debug message
$ tail test.log*
==> test.log <==
2021-11-28 18:48:38,520-simpleExample - CRITICAL - critical
message
==> test.log.1 <==
2021-11-28 18:48:38,519 - simpleExample - ERROR - error
message
==> test.log.2 <==
2021-11-28 18:48:38,518 - simpleExample - WARNING - warn
message
==> test.log.3 <==
2021.11.28 8:48:38,517 - simpleExample - INFO - info message
```

In the next example, let's see how to create a dictionary of configuration information and pass it to the *dictConfig()* function. This is simple and can be done using the *yaml* format as configuration. It creates a logger with the same module name, a console handler with the debug level set and a formatter with time, module name, level and message. The *dictConfig* expects input to be in the *dict* object, so that *dict* can be defined and the logger method called.

```
dict_filelog.py

import logging
import logging.config
import yaml
with open('dict_config.yaml', 'r') as f:
    config = yaml.safe_load(f.read())
    logging.config.dictConfig(config)
logger = logging.getLogger(__name__)
logger.debug('This is a debug message')
logger.info('This is a info message')
logger.error('This is a error message')
```